01 | 맵을 만드는 방식

공개 지도 윤곽을 3D 형상으로 만들고, 부족한 정보는 추정·가상 생성으로 보완합니다. 지도 위치, 높이, 외관의 출처와 정확도는 서로 다릅니다.

데이터에서 화면까지

OpenStreetMap 스냅샷 → 위·경도 평면 변환 → 1.2 km 범위 절단 → 건물 돌출·도로 생성 → 재질·조명 적용 → WebGL 렌더링

기준점은 서울 서초구 강남대로30길 7의 주소 좌표입니다. 원본의 건물 다각형은 높이 방향으로 돌출(Extrusion)하고, 도로의 연속 점은 폭을 가진 선분으로 바꿉니다. 공원과 하천은 별도 표면으로 표현합니다. 네이버·구글의 3D 타일이나 지도 화면을 가져온 방식은 아닙니다.

요소	V5.4 코드에서 확인한 생성 방식
건물 윤곽 427개	OSM 건물 다각형을 사용. 표시 영역은 기준점 주변 X·Z 각각 -600~+600으로 절단.
높이 태그 기반 188개	height 태그 우선. 없으면 building:levels × 3.3 m. 층수 환산은 가정이며 실측 높이와 구분.
높이 추정 239개	높이·층수가 없으면 OSM ID에 따라 12~35.1 m의 반복 가능한 추정값 사용.
가상 건물 183개	빈 공간을 30 단위 격자로 검사. 기존 건물·도로·공원·하천 주변과 기준점을 피해 추가. 전체 610개.
도로·하천 / 토지	도로·하천 way 344개, 공원·수면 등 19개. 도로 폭은 태그, 차선 수 또는 도로 종류별 기본값 적용.
지면·외관	지면은 Y=14의 평면. 외벽·창문·옥상 설비·나무는 시각적 표현. DEM이나 실측 표고는 사용하지 않음.

렌더링 비용과 지글거림을 줄이는 처리

같은 재질의 건물·도로 형상을 병합해 드로 콜을 줄이고, 나무와 드론에는 인스턴싱을 사용합니다. 창문 무늬는 셰이더에서 생성하며 화면상 크기에 따라 필터링합니다. 도로·보도·차선의 높이와 깊이 오프셋을 구분하고 카메라 근평면을 조절해 겹친 표면의 깜빡임을 줄입니다.

심사위원에게 이렇게 설명

"실제 지역의 공개 건물·도로 윤곽을 바탕으로 3D 도시를 구성했습니다. 높이가 없는 건물과 빈 공간은 추정·가상 형상으로 보완했고, 실행에 필요한 지도 데이터는 로컬에 저장합니다."

확인 파일: city-data.js, city-world.js, city-fill.js, city-spatial.js, render-quality.js, MAP_SOURCES.txt. 지리 데이터: © OpenStreetMap contributors / ODbL 1.0. https://www.openstreetmap.org/copyright

02 | 좌표계와 시점 변환

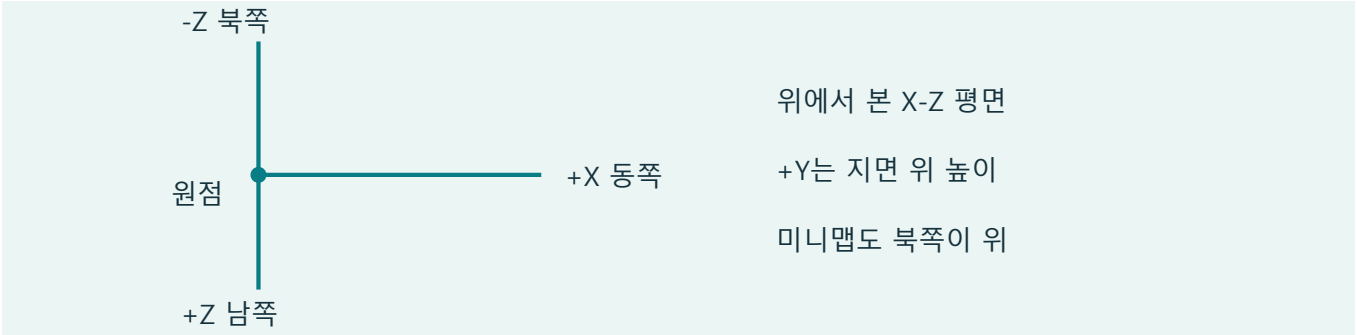
위·경도를 그대로 3D 좌표로 사용하지 않고, 대회장 주소를 원점으로 한 미터 규모의 지역 평면 좌표로 변환합니다. Three.js 장면은 오른손 좌표계를 사용합니다.

항목	정의
수평 원점	위도 φ_0=37.4819231°, 경도 λ_0=127.0375193°
축 방향	+X: 동쪽 / +Y: 위쪽 / +Z: 남쪽. 북쪽은 -Z.
단위와 기준 높이	수평 1 좌표 단위는 약 1 m. 지면은 Y=14이며 해발 14 m라는 뜻은 아님.
표시 범위	X, Z 각각 -600~+600. 약 1.2 km × 1.2 km.

코드에 사용된 평면 근사식

$$X = (\lambda - \lambda_0) \times 111{,}320 \times \cos(\varphi_0)$$
$$Z = -(\varphi - \varphi_0) \times 111{,}320$$

위·경도 차이는 도(degree) 단위입니다. cos 계산에는 기준 위도를 라디안으로 바꿔 넣습니다. 결과는 소수 둘째 자리까지 저장합니다. 작은 지역을 표현하는 근사이며, UTM·ECEF 변환이나 정밀 측량용 지리 기준계 계산을 구현한 것은 아닙니다.



한 좌표를 여러 화면에 표현

전체 3D·추적·탑승: 원근 카메라로 표현합니다. 추적 시점은 선택한 드론 뒤쪽에 카메라를 두고, 탑승 시점은 드론 위치 근처에서 전방을 봅니다. 객체의 월드 좌표는 유지하고 카메라 위치·방향만 변경합니다.

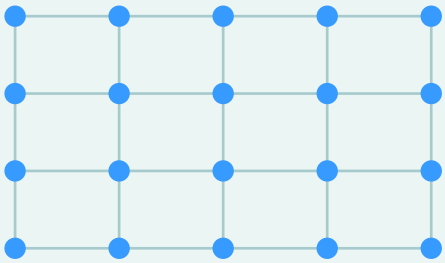
2D 지도: 직교 카메라를 (0, 1200, 0)에 두고 아래를 봅니다. 화면 위쪽을 -Z로 맞춥니다. 미니맵은 화면 중심 c와 축척 s에 대해 화면 X=c+X·s, 화면 Y=c+Z·s로 투영하므로 북쪽이 위에 표시됩니다.

센서 입력: NU40의 각속도와 표본 시간 차이로 회전량을 계산하여 가상 카메라의 yaw·pitch에 반영합니다. IMU를 지도상의 절대 위치나 GPS 좌표로 변환하는 기능은 아닙니다.

확인 파일: tmp/build_v54_city_data.py, city-data.js, app.js, hud.js, imu-direction.js. 지면 높이와 카메라 좌표는 화면 표현용 기준입니다.

03 | 드론을 격자로 배치한 이유

중앙의 작은 원에 몰려 보이던 문제를 해결하기 위해 파란 합성 드론을 최대 20개의 5열×4행 슬롯으로 분산합니다. 슬롯을 고정하고 전체에 같은 회전을 적용해 배치가 흐트러지지 않도록 했습니다.



5열 × 4행 / 최대 20 슬롯

간격 160 / 폭 640 × 깊이 480

회전 속도 0.006 rad/s

새 드론은 빈 슬롯에 5초 합류

수학적 근거: 공통 회전은 거리를 보존

슬롯 번호 k=0…19, 열 c=k mod 5, 행 r=floor(k/5)라 두면 초기 위치는 다음과 같습니다.
$x_0=160(c-2)$, $z_0=160(r-1.5)$

시간 t에서 θ=0.006t로 두고 모든 슬롯에 동일하게 적용합니다.
$x=x_0\cos\theta-z_0\sin\theta$
$z=x_0\sin\theta+z_0\cos\theta$

회전행렬 R은 $R^TR=I$를 만족하므로 $\|R(q_i-q_j)\|=\|q_i-q_j\|$입니다. 따라서 합류가 끝난 자동 배치에서는 최소 슬롯 간격 160이 유지됩니다. 바깥 슬롯의 반경은 400이고 속도는 400×0.006=2.4 좌표 단위/초입니다. 한 바퀴는 약 17분 27초입니다.

기존 드론을 흔들지 않는 5초 합류

기존 슬롯 번호는 유지하고 새 드론에 빈 슬롯만 배정합니다. 합류 진행률 u=clamp(경과시간/5, 0, 1), $s=3u^2-2u^3$로 두어 다음과 같이 이동합니다.
표시 위치 = (1-s)×합류 시작 위치 + s×현재 슬롯 위치

현재 코드의 최고 건물 높이는 108이며, 파란 대형은 Y=169에 표시합니다. 지면 Y=14와 비교하면 높이 155로, 최고 지붕보다 47 위입니다. 이 값은 가상 장면의 가독성을 위한 기준입니다.

설계 이유와 적용 범위

격자는 중앙까지 공간을 고르게 쓰고, 드론 간 분리와 객체 선택을 쉽게 만듭니다. 빈 슬롯만 채우므로 기존 객체가 갑자기 재배치되지 않습니다. 간격·속도·고도는 시연용 설계값이며, 최적 탐지 범위나 실제 비행 안전거리를 산출한 값은 아닙니다. 합류 중·수동 이동·게임 이벤트에는 격자 간격 보장이 적용되지 않습니다.

확인 파일: formation.js, formation.test.js, city-spatial.js, scenario.js. 본 문서는 2026-10-11 로컬 V5.4 코드와 데이터 기준이며, 실기체 운용·비행 제어 문서가 아닙니다.